

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЗАПОРІЗЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ

МАТЕМАТИЧНИЙ ФАКУЛЬТЕТ

Кафедра програмної інженерії

КУРСОВА РОБОТА

з

Бази даних

(назва дисципліни)

на тему: «База даних оцінок відеоігор»

Студента _____ 2 _____ курсу, групи 6.1214-пі2
спеціальності _____ 121 інженерія програмного забезпечення
(шифр і назва спеціальності)

О.О. Кривокоритов

(ініціали та прізвище)

Керівник

Климчук Анастасія Олександрівна

(посада, вчене звання, науковий ступінь, прізвище та ініціали)

Національна шкала: _____

Кількість балів: _____

Оцінка ECTS: _____

Члени комісії:

(підпис)

(ініціали та прізвище)

(підпис)

(ініціали та прізвище)

(підпис)

(ініціали та прізвище)

Запоріжжя – 2026

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЗАПОРІЗЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ

Факультет математичний

Кафедра програмної інженерії

Дисципліна Бази даних

Спеціальність 121 інженерія програмного забезпечення

(шифр і назва)

Освітня програма Програмна інженерія

З А В Д А Н Н Я
НА КУРСОВУ РОБОТУ СТУДЕНТОВІ (СТУДЕНТЦІ)

Кривокоритову Олександру Олександровичу

(прізвище, ім'я та по-батькові)

- | | |
|--|--|
| 1. Тема роботи | <u>«База даних оцінок відеоігор»</u> |
| <hr/> | |
| 2. Строк здачі студентом закінченої роботи | <u>21.05.2026 р.</u> |
| <hr/> | |
| 3. Вихідні дані до роботи | <u>1. Постановка задачі.</u> |
| | <u>2. Перелік літератури.</u> |
| <hr/> | |
| 4. Зміст роботи | |
| | <u>1. Постановка задачі.</u> |
| | <u>2. Основні теоретичні відомості.</u> |
| | <u>3. Система управління базою даних</u> |
| <hr/> | |
| 5. Перелік графічного матеріалу | |
| | <u>презентація</u> |
| <hr/> | |
| 6. Дата видачі завдання | <u>17.03.2025 р.</u> |

КАЛЕНДАРНИЙ ПЛАН

№	Назва етапів курсової роботи	Термін виконання етапів роботи	Примітка
1.	Розробка плану роботи	29.03.2026	
2.	Збір вихідних даних	04.04.2026	
3.	Обробка методичних джерел	05.04.2026	
4.	Розробка FrontEnd	02.05.2026	
5.	Розробка BackEnd	15.05.2026	
6.	Оформлення курсової роботи	20.05.2026	
7.	Захист курсової роботи	21.05.2026	

Студент

(підпис)

О.О. Кривокоритов

(ініціали та прізвище)

Керівник роботи

(підпис)

А. О. Климчук

(ініціали та прізвище)

РЕФЕРАТ

Курсова робота «Розробка бази даних ігрової інформаційної системи з використанням СУБД MySQL»: 34 с., 8 рис., 2 табл., 7 джерел.

СУБД, MYSQL, РЕЛЯЦІЙНА БАЗА ДАНИХ, SQL, PYTHON, FLASK, ТРИГЕРИ, ЗБЕРЕЖЕНІ ПРОЦЕДУРИ, ФРОНТЕНД, БЕКЕНД, АНАЛІТИКА.

Об'єкт дослідження — процес розробки та впровадження бази даних ігрової інформаційної системи з використанням реляційної СУБД MySQL.

Предмет дослідження — клієнт-серверна архітектура веб-застосунку, механізми бекенду (Flask, сесії, безпека), просунуті інструменти СУБД MySQL (тригери, збережені процедури, представлення) та засоби проектування фронтенду.

Мета роботи — аналіз предметної області, проектування реляційної моделі даних та програмна реалізація клієнтської і серверної частин ігрової інформаційної системи «Game Review System».

Методи дослідження — методи реляційного моделювання (3НФ), об'єктно-орієнтоване проектування (паттерн MVC), розробка користувацьких інтерфейсів та серверної логіки веб-застосунків.

У роботі наведено порівняльний аналіз реляційних та нереляційних систем управління базами даних.

В першому розділі приділено увагу проектуванню клієнтської частини (фронтенду) інформаційної системи «Портал оглядів відеоігор». Наведено опис предметної області та розроблено структуру інтерфейсу користувача за допомогою Jinja2-шаблонів, CSS-фреймворку Tailwind CSS та компонентів Flowbite. Окремо описано інтеграцію JS-бібліотек ApexCharts (для інтерактивних графіків статистики) та Simple-DataTables (для відображення логів адміністратора). Структуру даних візуалізовано ER-діаграмою.

В другому розділі описано розробку серверної логіки (бекенду) на базі мікрофреймворку Flask та мови Python. Детально розібрано підключення до

бази даних MySQL, реалізацію збережених процедур для безпечного додавання даних, тригерів для автоматичного ведення логів аудиту в БД, а також використання віконних функцій для розрахунку рейтингів ігор. Розглянуто механізми авторизації, сесій та безпеки користувачів.

Результати роботи можуть бути використані для подальшого проектування та розробки інтерактивних веб-порталів із чітким розділенням на фронтенд та бекенд.

ЗМІСТ

Завдання на курсову роботу	2
Реферат	4
Вступ.....	6
1 ПРОЕКТУВАННЯ БАЗИ ДАНИХ	8
1.1 Аналіз існуючих СУБД	8
1.2 Опис предметної області.....	11
1.3 Побудова ER-діаграм.....	15
2 РЕАЛІЗАЦІЯ БАЗИ ДАНИХ ІНФОРМАЦІЙНОЇ СИСТЕМИ «База даних оцінок відеоігор» З ВИКОРИСТАННЯМ СУБД MySQL.....	17
2.1 Генерування колекцій у MySQL	17
2.2 Програмування запитів.....	19
2.3 Розробка клієнтської програми	21
ВИСНОВКИ.....	29
ДОДАТОК А Програмна реалізація бази даних	32

ВСТУП

У сучасну цифрову епоху індустрія відеоігор перетворилася на один із найбільших глобальних секторів розваг, що щодня генерує колосальні обсяги інформації. Ефективне керування, структурування та аналіз даних про ігрові релізи, розробників, видавців та користувацький досвід потребує створення спеціалізованих інформаційних систем. Традиційні підходи до збереження даних без використання реляційних зв'язків не здатні забезпечити швидкий пошук, цілісність інформації та гнучку аналітику рейтингів у реальному часі. Тому розробка оптимізованої бази даних та пов'язаного з нею веб-інтерфейсу є критично важливою для автоматизації накопичення відгуків, оцінювання контенту та надання інтерактивної звітності користувачам.

Отже, актуальним завданням є розробка бази даних ігрової інформаційної системи «Game Review System» з використанням СУБД MySQL.

Метою курсової роботи є розробка веб-орієнтованої інформаційної системи для каталогізації відеоігор, яка дозволяє систематизувати дані про ігрову індустрію, забезпечити збір відгуків та оцінок користувачів, а також автоматизувати адміністрування та моніторинг системи.

Задачі, які необхідно розв'язати для досягнення поставленої мети:

проаналізувати існуючі системи управління базами даних, здійснивши порівняльний аналіз реляційних та нереляційних СУБД;

проаналізувати предметну область ігрової індустрії та сформулювати вимоги до інформаційної системи;

спроектувати концептуальну схему (ER-діаграму) та фізичну реляційну модель бази даних, обґрунтувати її відповідність нормальним формам;

реалізувати базу даних засобами СУБД MySQL (включаючи створення тригерів, збережених процедур та віконних функцій);

розробити клієнтську частину (інтерфейс користувача) засобами шаблонізатора Jinja2, фреймворку Tailwind CSS, Flowbite та графічних модулів ApexCharts;

реалізувати серверну логіку (бекенд) засобами мови Python та мікрофреймворку Flask;

протестувати розроблену інформаційну систему та перевірити роботу механізмів безпеки і розмежування ролей.

Об'єкт дослідження - процес розробки бази даних та клієнт-серверної архітектури ігрової інформаційної системи.

Предмет дослідження - СУБД MySQL, мікрофреймворк Flask, технології верстки Tailwind CSS та методи забезпечення цілісності даних на рівні СУБД.

Методи дослідження - методи реляційного моделювання, теорія нормалізації баз даних, об'єктно-орієнтоване проектування та системний аналіз.

Структурно курсова робота складається з таких компонентів: вступ, 2 розділи, висновки, додатки, перелік посилань із 10 найменувань.

Перший розділ присвячений порівняльному аналізу реляційних та NoSQL СУБД, аналізу предметної області, розробці клієнтської частини (фронтенду) за допомогою Tailwind CSS, Flowbite, інтерактивних графіків ApexCharts, а також концептуальному проектуванню бази даних та її нормалізації до третьої нормальної форми (3НФ).

Другий розділ описує реалізацію серверної частини (бекенду) застосунку на базі Flask та інтеграцію з СУБД MySQL. У ньому розглядаються питання підключення до БД, створення збережених процедур для додавання даних, тригерів для системного аудиту логів (відображуваних через SimpleDataTables), використання віконних функцій (RANK()) для розрахунку рейтингів, а також налаштування сесій користувачів та безпеки.

У додатках наведено схему бази даних, діаграму класів та вихідний програмний код.

1 ПРОЄКТУВАННЯ БАЗИ ДАНИХ

1.1 Аналіз існуючих СУБД

Вибір системи управління базами даних (СУБД) є першочерговим та визначальним етапом проектування будь-якої інформаційної системи. СУБД визначає не лише спосіб фізичного збереження інформації, але й механізми забезпечення цілісності даних, швидкість виконання запитів та загальну архітектуру програмного забезпечення [1].

У сучасній практиці розробки інформаційних ресурсів найчастіше постає дилема вибору між класичною реляційною моделлю даних та гнучкими нереляційними рішеннями класу NoSQL [2]. Для обґрунтування вибору технологічної платформи інформаційної системи «База даних оцінок відеоігор» було проведено аналіз трьох популярних систем: реляційної СУБД MySQL та документоорієнтованих NoSQL СУБД MongoDB і Apache CouchDB.

Реляційна СУБД MySQL

MySQL є реляційною системою управління базами даних з відкритим вихідним кодом. Вона базується на мові структурованих запитів SQL та використовує табличну модель представлення даних, де зв'язки між сутностями реалізуються за допомогою первинних і зовнішніх ключів [3]. Головною перевагою є суворе дотримання вимог ACID, декларативна цілісність через зовнішні ключі з правилами каскадного видалення (ON DELETE CASCADE), а також наявність потужних інструментів програмування на рівні бази даних: збережених процедур, тригерів та віконних функцій. Слабкою стороною є необхідність жорсткого проектування схеми та складність горизонтального масштабування [4].

Документоорієнтована СУБД MongoDB

MongoDB представляє сімейство нереляційних баз даних і використовує документоорієнтований підхід, де дані зберігаються у гнучкому JSON-подібному форматі BSON [5]. Вона пропонує повну гнучкість схеми (Schema-less) та високу швидкість обробки неструктурованих потоків даних. Проте в MongoDB відсутня декларативна цілісність даних на рівні ядра СУБД (немає зовнішніх ключів), а операції об'єднання даних (аналог JOIN) є ресурсомісткими та вимагають дублювання інформації на рівні колекцій [2].

Документоорієнтована СУБД Apache CouchDB

CouchDB – це документоорієнтована база даних, яка фокусується на простоті використання, надійності та веб-орієнтованості, використовуючи HTTP REST API для взаємодії [6]. Вона пропонує надійну синхронізацію даних між розподіленими серверами (Master-Master реплікація). Слабкою стороною системи є використання концепції MapReduce для генерації представлень, що значно ускладнює розробку динамічних аналітичних запитів та звітів у реальному часі [6].

Для детального порівняння обраних систем було сформовано таблицю порівняльного аналізу (табл. 1.1).

Таблиця 1.1 – Порівняльний аналіз систем управління базами даних

Критерій порівняння	MySQL	MongoDB	CouchDB
Модель даних	Реляційна (таблиці, зв'язки)	Документоорієнтована (BSON)	Документоорієнтована (JSON)
Мова запитів	SQL (декларативна)	MQL (специфічний API)	HTTP REST API / MapReduce
Цілісність даних	Суворе дотримання	Часткова підтримка ACID, рівні	Транзакції на документа,

Критерій	MySQL	MongoDB	CouchDB
порівняння	L		
	ACID, зовнішні ключі	відсутні ключі	зовнішні відсутні ключі
	Ефект		
Операції зв'язування	ивне виконання (оператори J OIN)	Обмежене виконання (оператор \$lookup)	Відсутнє на рівні ядра СУБД (через MapReduce)
	Збережені		
Програмування в БД	процедури, тригери, віконні функції	Відсутнє (за виключенням серверного JS)	(за Представлення MapReduce, функції валідації
Масштабованість	Переважно вертикальна	Горизонтальна (автоматичний шардинг)	Горизонтальна (Master-Master реплікація)

Обґрунтування вибору СУБД для проекту

Аналіз показав, що для інформаційної системи «База даних оцінок відеоігор» критично важливими є суворі несуперечливість даних (кожен відгук має бути жорстко прив'язаний до існуючої гри та користувача) та можливість реалізації бізнес-логіки безпосередньо в базі даних.

NoSQL рішення (MongoDB, CouchDB) орієнтовані на збереження великих обсягів слабоструктурованих даних, проте не гарантують каскадної цілісності та вимагають дублювання даних для уникнення операцій з'єднання.

Тому для розробки було обрано **СУБД MySQL**. Її використання дозволить побудувати повністю нормалізовану реляційну модель (3НФ),

використати збережену процедуру (`sp_add_game_full`) для безпечного додавання даних, тригер (`tr_after_review_insert`) для ведення логів аудиту на рівні СУБД та віконні функції (`RANK()`) для розрахунку рейтингів ігор у реальному часі.

1.2 Опис предметної області

Предметною областю курсової роботи є ігрова індустрія, а саме — інформаційні процеси, пов'язані з обліком відеоігор, їхніх характеристик (розробники, видавці, жанри, гральні платформи) та накопиченням користувацьких оцінок і текстових відгуків. Розроблена система покликана вирішити проблему пошуку ігор, агрегації рейтингів та забезпечення прозорого адміністрування ресурсу.

Користувачі системи та їхня кількість

Система орієнтована на обслуговування трьох основних груп користувачів:

- **Гості (неавторизовані відвідувачі):** це найчисленніша категорія користувачів (потенційно тисячі відвідувачів на день). Вони використовують систему як інформаційний довідник: здійснюють пошук та фільтрацію ігор, переглядають інформацію про розробників, читають відгуки інших людей та аналізують загальну статистику і рейтингів ігор;
- **Авторизовані користувачі:** активні геймери (кількість може вимірюватися сотнями чи тисячами зареєстрованих облікових записів). Окрім можливостей гостей, вони мають право створювати власний контент — залишати оцінки іграм у діапазоні від 0 до 10 та писати текстові рецензії;
- **Адміністратор:** спеціалізований користувач з повними правами доступу (у реальних умовах це 1–2 особи). Адміністратор відповідає за

наповнення бази даних контентом (додавання нових ігор, описів, обкладинок) та здійснює технічний моніторинг системи шляхом перегляду журналів аудиту дій користувачів.

Задачі, які виконує система

Для автоматизації бізнес-процесів предметної області розроблена система виконує такі ключові задачі:

- забезпечення безпечної реєстрації користувачів із хешуванням паролів та автентифікації на основі сесій;
- динамічний пошук ігор за назвою, фільтрація за жанрами та платформами;
- агрегація оцінок користувачів та розрахунок середнього рейтингу ігор у реальному часі;
- захищене додавання ігрового контенту через веб-інтерфейс адміністратора;
- автоматичне протоколювання дій користувачів на рівні бази даних для запобігання накруткам оцінок;
- візуалізація аналітичних даних (кількість ігор по жанрах, активність користувачів) у вигляді інтерактивних графіків.

Для структурування інформації предметної області було виділено основні сутності та їхні атрибути, які представлені у таблиці 1.2.

Таблиця 1.2 – Опис сутностей та атрибутів предметної області

Сутність	Опис сутності	Атрибути сутності
Користувачі	Зареєстровані користувачі системи, які мають право оцінювати ігри.	– Унікальний код; – Ім'я користувача (логін); – Електронна пошта; – Хеш пароля;

Сутність	Опис сутності	Атрибути сутності
		– Дата реєстрації; – Ознака прав адміністратора.
Ігри	Відеоігри, що містяться в каталозі інформаційної системи.	– Унікальний код гри; – Назва гри; – Дата релізу; – Текстовий опис; – Код розробника (зовнішній ключ); – Код видавця (зовнішній ключ); – Посилання на обкладинку.
Відгуки	Оцінки та рецензії, залишені користувачами до конкретних ігор.	– Унікальний код відгуку; – Код гри (зовнішній ключ); – Код користувача (зовнішній ключ); – Оцінка (від 0 до 10); – Текст відгуку; – Дата створення.
Розробники	Компанії-розробники, що безпосередньо створили гру.	– Унікальний код розробника; – Назва студії;

Сутність	Опис сутності	Атрибути сутності
		– Країна реєстрації студії.
Видавці	Компанії, що здійснюють видання та фінансування ігор.	– Унікальний код видавця; – Назва компанії-видавця; – Країна реєстрації компанії.
Жанри	Жанрова класифікація відеоігор (наприклад: RPG, Action, Shooter).	– Унікальний код жанру; – Назва жанру.
Платформи	Ігрові платформи, на яких доступна гра (наприклад: PC, PS5, Xbox).	– Унікальний код платформи; – Повна назва платформи; – Скорочена назва (аббревіатура).

Типи зв'язків між сутностями

Між виділеними сутностями існують такі типи реляційних зв'язків:

- Зв'язок «**один-до-багатьох**» (1:N) між сутностями *Розробники* та *Ігри* (одна студія може розробити багато ігор, але гра створюється однією конкретною студією);
- Зв'язок «**один-до-багатьох**» (1:N) між сутностями *Видавці* та *Ігри* (один видавець фінансує багато ігор, але гра має одного офіційного видавця);
- Зв'язок «**один-до-багатьох**» (1:N) між сутностями *Користувачі* та *Відгуки* (один користувач може написати багато відгуків, але кожен відгук належить тільки одному автору);

- Зв'язок «один-до-багатьох» (1:N) між сутностями *Ігри* та *Відгуки* (одна гра може отримати безліч відгуків від різних людей, але кожен відгук стосується лише однієї конкретної гри);
- Зв'язок «багато-до-багатьох» (M:N) між сутностями *Ігри* та *Жанри* (одна гра може поєднувати кілька жанрів, і до одного жанру належить багато ігор; реалізується через допоміжну таблицю `game_genres`);
- Зв'язок «багато-до-багатьох» (M:N) між сутностями *Ігри* та *Платформи* (гра може вийти на різних платформах, і кожна платформа містить каталог з багатьох ігор; реалізується через таблицю `game_platforms`).

1.3 Побудова ER-діаграм

Логічна структура бази даних «База даних оцінок відеоігор» представлена у вигляді ER-діаграми на рисунку 1.1.

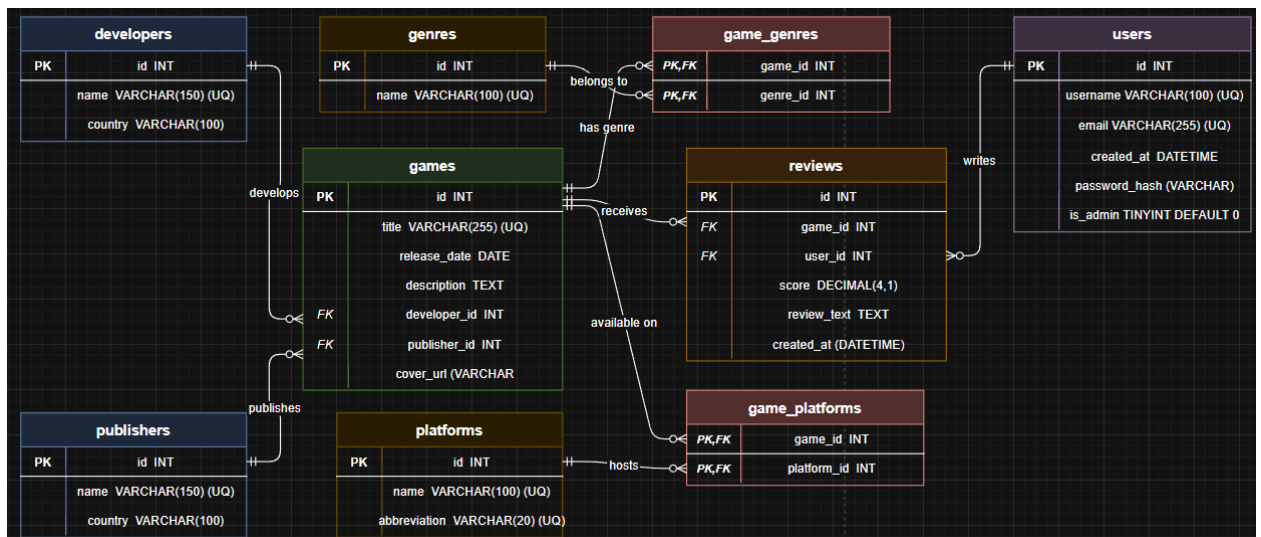


Рисунок 1.1 – ER-діаграма сутностей та зв'язків моделі бази даних «База даних оцінок відеоігор».

Таблиця **users** (Користувачі) зберігає облікові дані користувачів та містить поля `id` (первинний ключ PK, тип INT), `username` та `email` (тип VARCHAR, унікальні), `password_hash` (VARCHAR), `is_admin` (TINYINT) та `created_at` (DATETIME).

Таблиця **games** (Ігри) призначена для збереження інформації про ігри та складається з полів `id` (PK, INT), `title` (VARCHAR, унікальне), `release_date` (DATE), `description` (TEXT), `cover_url` (VARCHAR) та зовнішніх ключів `developer_id` і `publisher_id` (FK, INT).

Таблиця **reviews** (Відгуки) об'єднує оцінки та текстові рецензії користувачів за допомогою полів `id` (PK, INT), `score` (DECIMAL), `review_text` (TEXT), `created_at` (DATETIME) та зовнішніх ключів `game_id` і `user_id` (FK, INT).

Таблиці **developers** та **publishers** містять інформацію про компанії, де `id` є первинним ключем, а назва компанії (`name`) та її країна (`country`) записуються в полях типу VARCHAR. Таблиці **genres** та **platforms** слугують довідниками категорій.

Зв'язки типу один-до-багатьох (1:N) пов'язують таблиці розробників та видавців з таблицею ігор, де встановлено обмеження ON DELETE RESTRICT для збереження цілісності даних. Зв'язки користувачів та ігор з відгуками використовують каскадне видалення ON DELETE CASCADE, що автоматично очищує базу від пов'язаних відгуків у разі видалення основної сутності.

Зв'язки типу багато-до-багатьох (M:N) між іграми, жанрами та платформами реалізовано через проміжні таблиці `game_genres` та `game_platforms` зі складеними первинними ключами. Спроектowana схема даних відповідає вимогам третьої нормальної форми (3НФ), що мінімізує дублювання інформації.

2 РЕАЛІЗАЦІЯ БАЗИ ДАНИХ ІНФОРМАЦІЙНОЇ СИСТЕМИ «База даних оцінок відеоігор» .3 ВИКОРИСТАННЯМ СУБД MySQL

2.1 Генерування колекцій у MySQL

Для реалізації спроектованої схеми бази даних використовується реляційна СУБД MySQL Community Server. Встановлення та налаштування середовища виконується шляхом інсталяції сервера бази даних та утиліти адміністрування MySQL Workbench. Після встановлення створюється локальне з'єднання з сервером (localhost) на стандартному порті 3306. Конфігураційні параметри підключення, такі як хост, користувач, пароль та назва бази даних, виносяться в ізольований файл налаштувань бекенд-додатка, що запобігає жорсткому кодуванню облікових даних у коді програми.

Процес ініціалізації бази даних та створення таблиць (колекцій) виконується за допомогою SQL-скрипту, який містить DDL-запити для створення структури відношень, визначення первинних і зовнішніх ключів та каскадних обмежень цілісності даних. Нижче наведено SQL-код для створення основних таблиць бази даних інформаційної системи.

```
CREATE TABLE IF NOT EXISTS developers (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(150) NOT NULL UNIQUE,  
    country VARCHAR(100) NOT NULL  
);  
  
CREATE TABLE IF NOT EXISTS publishers (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(150) NOT NULL UNIQUE,  
    country VARCHAR(100) NOT NULL  
);  
  
CREATE TABLE IF NOT EXISTS genres (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL UNIQUE  
);  
  
CREATE TABLE IF NOT EXISTS platforms (  
    id INT AUTO_INCREMENT PRIMARY KEY,
```

```

name VARCHAR(100) NOT NULL UNIQUE,
abbreviation VARCHAR(20) NOT NULL UNIQUE
);

```

Рисунок 2.1 – Запити на створення довідкових таблиць бази даних

Таблиці розробників, видавців, жанрів та платформ створюються першими, оскільки вони є незалежними сутностями і їхні ідентифікатори використовуються як зовнішні ключі в інших таблицях. Наступним кроком виконується створення таблиць користувачів, ігор та відгуків, які безпосередньо зв'язують дані між собою.

```

CREATE TABLE IF NOT EXISTS users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(100) NOT NULL UNIQUE,
    email VARCHAR(255) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    is_admin TINYINT DEFAULT 0,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS games (
    id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(255) NOT NULL UNIQUE,
    release_date DATE NOT NULL,
    description TEXT NOT NULL,
    developer_id INT NOT NULL,
    publisher_id INT NOT NULL,
    cover_url VARCHAR(255),
    FOREIGN KEY (developer_id) REFERENCES developers(id),
    FOREIGN KEY (publisher_id) REFERENCES publishers(id)
);

```

Рисунок 2.2 – Запити на створення таблиць користувачів та ігор

При створенні таблиці ігор для полів розробника та видавця застосовано обмеження ON DELETE RESTRICT (що діє за замовчуванням при відсутності інших вказівок), це запобігає випадковому видаленню компаній, чийі ігри наявні в системі. На завершальному етапі генеруються таблиці відгуків та сполучні таблиці для реалізації зв'язків багато-до-багатьох.

```

CREATE TABLE IF NOT EXISTS reviews (
    id INT AUTO_INCREMENT PRIMARY KEY,
    game_id INT NOT NULL,
    user_id INT NOT NULL,
    score DECIMAL(4,1) NOT NULL,
    review_text TEXT NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,

```

```

        UNIQUE KEY uq_reviews_game_user (game_id, user_id),
        FOREIGN KEY (game_id) REFERENCES games(id) ON DELETE CASCADE,
        FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
    );

CREATE TABLE IF NOT EXISTS game_genres (
    game_id INT NOT NULL,
    genre_id INT NOT NULL,
    PRIMARY KEY (game_id, genre_id),
    FOREIGN KEY (game_id) REFERENCES games(id) ON DELETE CASCADE,
    FOREIGN KEY (genre_id) REFERENCES genres(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS game_platforms (
    game_id INT NOT NULL,
    platform_id INT NOT NULL,
    PRIMARY KEY (game_id, platform_id),
    FOREIGN KEY (game_id) REFERENCES games(id) ON DELETE CASCADE,
    FOREIGN KEY (platform_id) REFERENCES platforms(id) ON DELETE CASCADE
);

```

Рисунок 2.3 – Запити на створення таблиць відгуків та зв'язків жанрів і платформ

Використання обмеження **ON DELETE CASCADE** для відгуків та зв'язків гарантує автоматичне очищення бази даних від застарілих посилань у разі видалення гри чи користувача, забезпечуючи підтримання реляційної цілісності інформації.

2.2 Програмування запитів

Робота з розробленою базою даних здійснюється шляхом написання SQL-запитів, які забезпечують виконання базових операцій маніпулювання даними, пошуку інформації за різними критеріями та формування аналітичної звітності. Для демонстрації функціональних можливостей бази даних було спроектовано серію запитів, що охоплюють процеси внесення даних, фільтрації за текстовими шаблонами, видалення записів та створення складних звітів за допомогою об'єднань.

Далі представлено синтаксис запитів для додавання нових записів, пошуку ігор за першими літерами назви та фільтрації за датою релізу.

```
-- Додавання нового користувача до системи
INSERT INTO users (username, email, password_hash)
VALUES ('new_gamer', 'gamer@email.com', 'pbkdf2:sha256:hash_value');

-- Пошук ігор, назва яких починається на літеру 'B'
SELECT id, title, release_date FROM games
WHERE title LIKE 'B%';

-- Вибірка ігор, випущених після 1 січня 2020 року
SELECT id, title, release_date FROM games
WHERE release_date >= '2020-01-01';
```

Рисунок 2.4 – Запити на додавання, пошук за літерою та фільтрацію за датою

Запит на додавання запису дозволяє створювати нові облікові записи з первинним шифруванням паролів. Пошук за допомогою оператора LIKE дозволяє реалізувати гнучкий автодопис ігор у веб-інтерфейсі, а фільтрація за датою релізу обмежує вибірку актуальними проектами.

Наступний блок запитів демонструє видалення інформації, об'єднання кількох таблиць для отримання детальної інформації про гру та підрахунок середнього балу.

```
-- Видалення відгуку за його унікальним ідентифікатором
DELETE FROM reviews WHERE id = 5;

-- Отримання інформації про гру з назвою розробника, видавця та середнім балом
SELECT g.title, d.name AS developer, p.name AS publisher, AVG(r.score) AS avg_rating
FROM games g
JOIN developers d ON g.developer_id = d.id
JOIN publishers p ON g.publisher_id = p.id
LEFT JOIN reviews r ON g.id = r.game_id
GROUP BY g.id, g.title, d.name, p.name;
```

Рисунок 2.5 – Запити на видалення записів та багатотабличне об'єднання

Запит на видалення відгуку автоматично декрементує кількість рецензій гри завдяки реляційним зв'язкам. Багатотабличне об'єднання (JOIN) та групування за первинним ключем та атрибутами забезпечують динамічний підрахунок середнього арифметичного балу гри на основі всіх оцінок користувачів.

Окрему увагу в роботі приділено автоматизації процесів та генеруванню звітів за допомогою просунутих інструментів СУБД MySQL. Для цього реалізовано збережену процедуру додавання гри, тригер для логування аудиту відгуків та аналітичне представлення з віконними функціями.

```
-- Виклик збереженої процедури для транзакційного додавання гри
CALL sp_add_game_full(
    'Bloodborne',
    '2015-03-24',
    'Dark fantasy Action RPG',
    1,
    1,
    '/static/covers/bloodborne.png'
);

-- Перегляд аналітичного звіту з рейтингом ігор на основі віконної функції RANK
SELECT global_rank, title, total_score, rating_status
FROM v_game_rankings;
```

Рисунок 2.6 – Виклики збереженої процедури та вибірка з аналітичного представлення

Виклик збереженої процедури `sp_add_game_full` гарантує транзакційну цілісність, автоматично додаючи гру до бази даних із прив'язкою до існуючих ідентифікаторів розробника та видавця. Аналітичний звіт на основі представлення `v_game_rankings` використовує віконну функцію `RANK() OVER (ORDER BY COALESCE(avg_rev.score, 0) DESC)` для динамічного визначення місця гри в загальному рейтингу системи, а також викликає користувацьку функцію для текстової інтерпретації середнього балу. Логування додавання відгуків виконується на фоновому рівні бази даних через тригер `tr_after_review_insert`, який автоматично робить запис у системний журнал аудиту без втручання коду Flask.

2.3 Розробка клієнтської програми

Програмний комплекс має чітко організовану структуру каталогів та файлів, яка забезпечує ізоляцію серверної логіки від статичного контенту та шаблонів відображення. Структура файлів розробленого проекту представлена на рисунку 2.7.

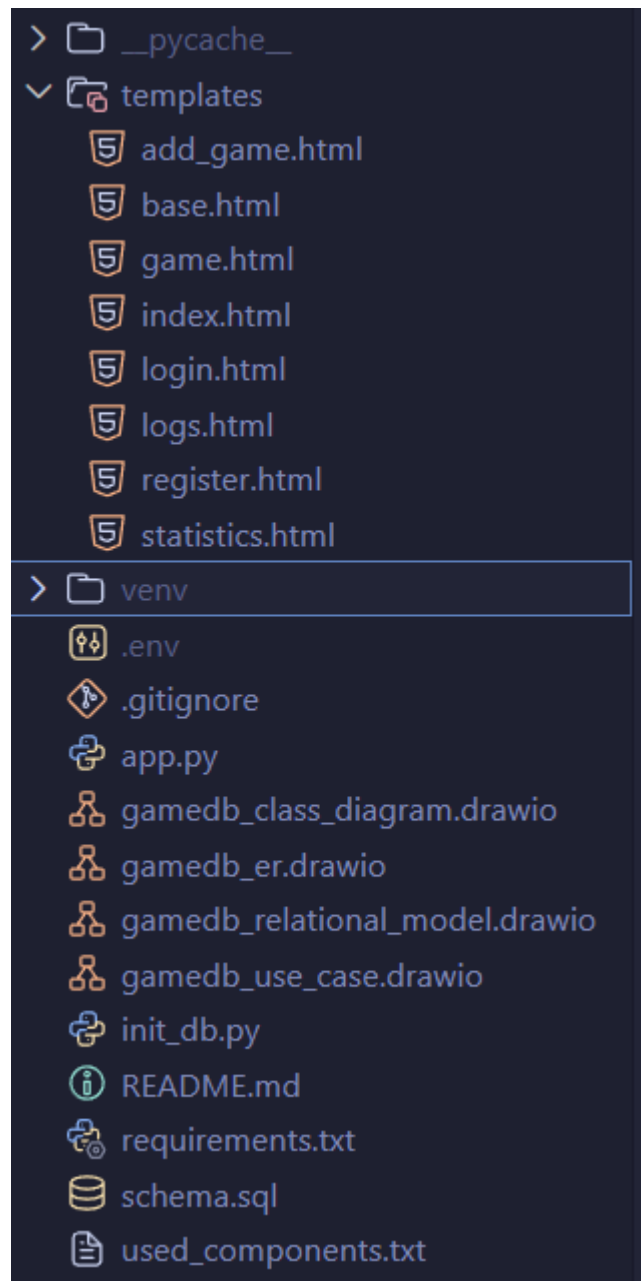


Рисунок 2.7 – Структура файлів та каталогів розробленого проекту.

Головним модулем веб-додатку є файл `app.py`, який виступає ядром системи, містить маршрутизацію та обробляє клієнтські запити. Файл `init_db.py` призначений для первинного розгортання та автоматичного наповнення бази даних тестовими даними на основі SQL-інструкцій з файлу `schema.sql`, де записано структуру таблиць, тригери та збережені процедури. Конфігураційний файл `.env` містить змінні оточення для безпечного підключення до сервера MySQL, а `requirements.txt` визначає перелік залежностей додатку. Всі шаблони сторінок інтерфейсу користувача згруповані в каталозі `templates`, а стилі, зображення обкладинок та скрипти графіків містяться в папці `static`.

Клієнтська програма ігрової інформаційної системи розроблена у вигляді сучасного веб-застосунку на основі клієнт-серверної архітектури та шаблону проектування MVC (Model-View-Controller). Як мову програмування бекенду обрано Python через його високу швидкість розробки та наявність потужних бібліотек для роботи з базами даних. Веб-інтерфейс реалізовано за допомогою мікрофреймворку Flask, який координує маршрутизацію запитів, обробляє сесії користувачів та забезпечує взаємодію з базою даних MySQL за допомогою драйвера mysql-connector-python.

Для візуального оформлення клієнтської частини (фронтенду) використано шаблонізатор Jinja2, CSS-фреймворк Tailwind CSS та бібліотеку компонентів Flowbite, що дозволило створити адаптивний темний інтерфейс. Інтерактивна аналітика та графіки побудовані за допомогою JS-бібліотеки ApexCharts, а пошук та пагінація логів аудиту реалізовані через модуль Simple-DataTables. Для опису функціональних вимог до системи з боку різних користувачів було побудовано діаграму варіантів використання (прецедентів), яка представлена на рисунку 2.8.

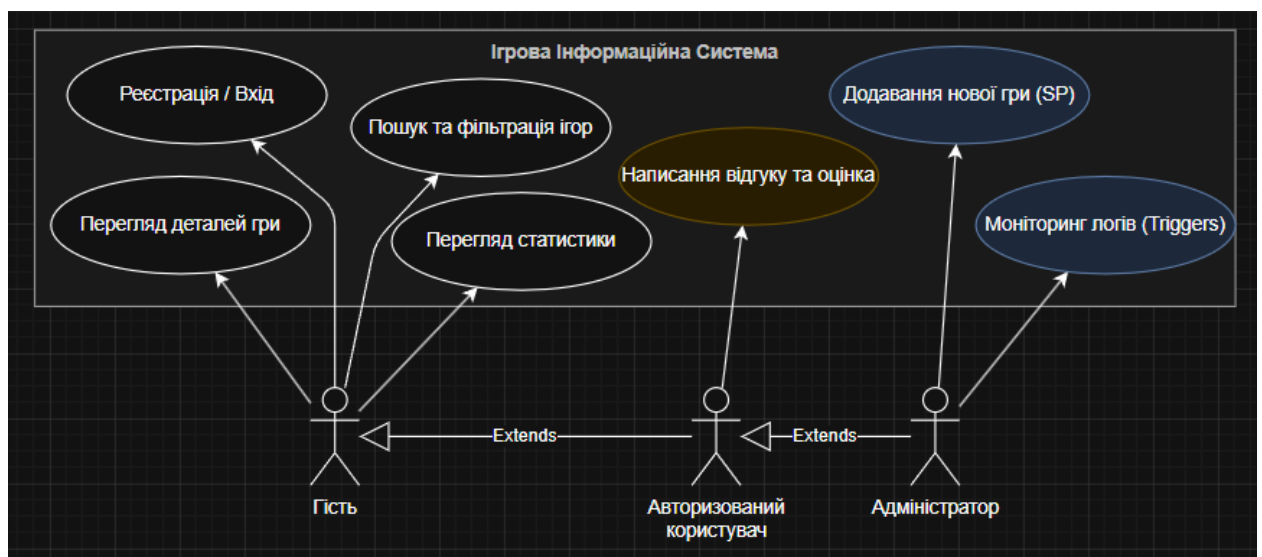


Рисунок 2.8 – Діаграма варіантів використання ігрової інформаційної системи.

Діаграма прецедентів виділяє три рівні права доступу користувачів. Неавторизований відвідувач (Гість) має доступ до базового перегляду каталогу ігор, пошуку, сортування та вивчення аналітичної статистики. Авторизований користувач успадковує всі можливості гостя та додатково отримує доступ до авторизованої зони, де може публікувати нові оцінки та писати рецензії на ігри. Адміністратор системи володіє ексклюзивними правами на додавання нових ігор, видавців і розробників через захищений інтерфейс, а також має доступ до сторінки системних логів для моніторингу дій користувачів.

Логічна структура класів та архітектурних компонентів додатку, що реалізує паттерн MVC, зображена на діаграмі класів на рисунку 2.9.

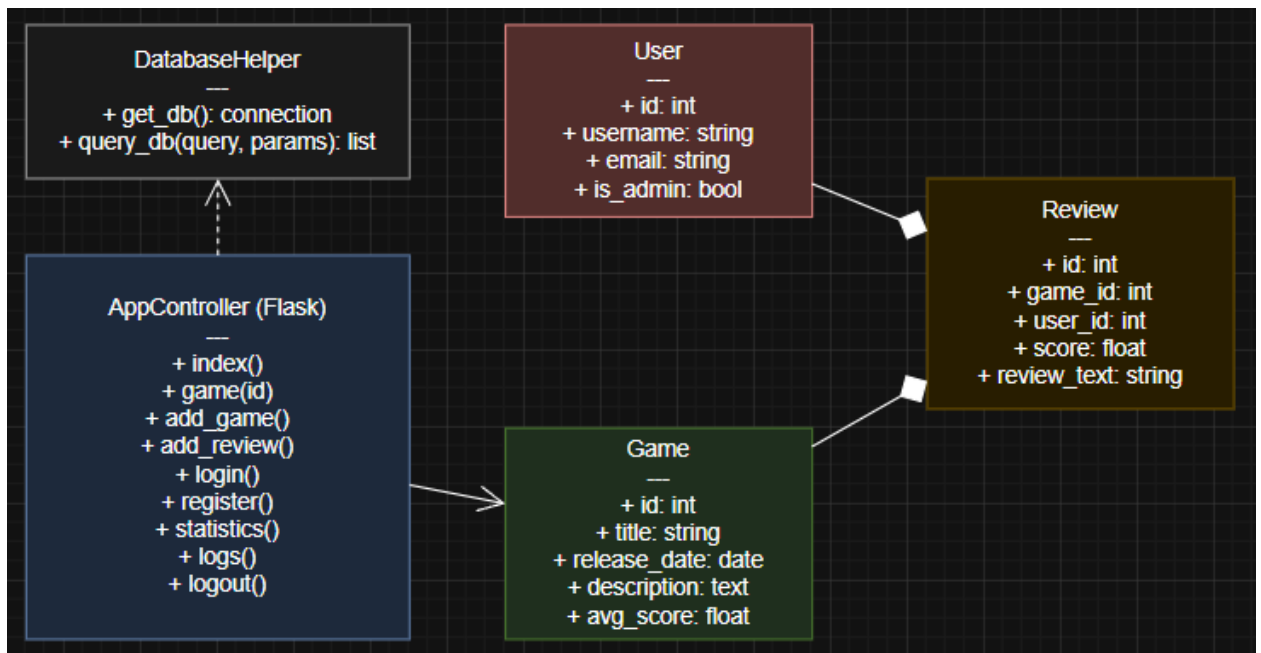


Рисунок 2.9 – Діаграма класів програмного застосунку.

Діаграма класів відображає взаємодію між трьома рівнями MVC архітектури застосунку. Модель даних представлена класом DatabaseHelper, який відповідає за підключення до MySQL та виконання запитів, а також сутностями Game, User та Review. Рівень контролера (Controller) реалізований класом ApplicationController, який містить методи обробки маршрутів Flask, такі як index, game_details, add_review та view_logs. Рівень представлення (View) складається з набору шаблонів Jinja2, які динамічно рендерить сервер Flask для відображення HTML-сторінок користувачу. Приклади роботи веб-інтерфейсу розробленого застосунку продемонстровано за допомогою скріншотів готової програми. На рисунку 2.10 зображено головну сторінку інформаційної системи «Game Review System».

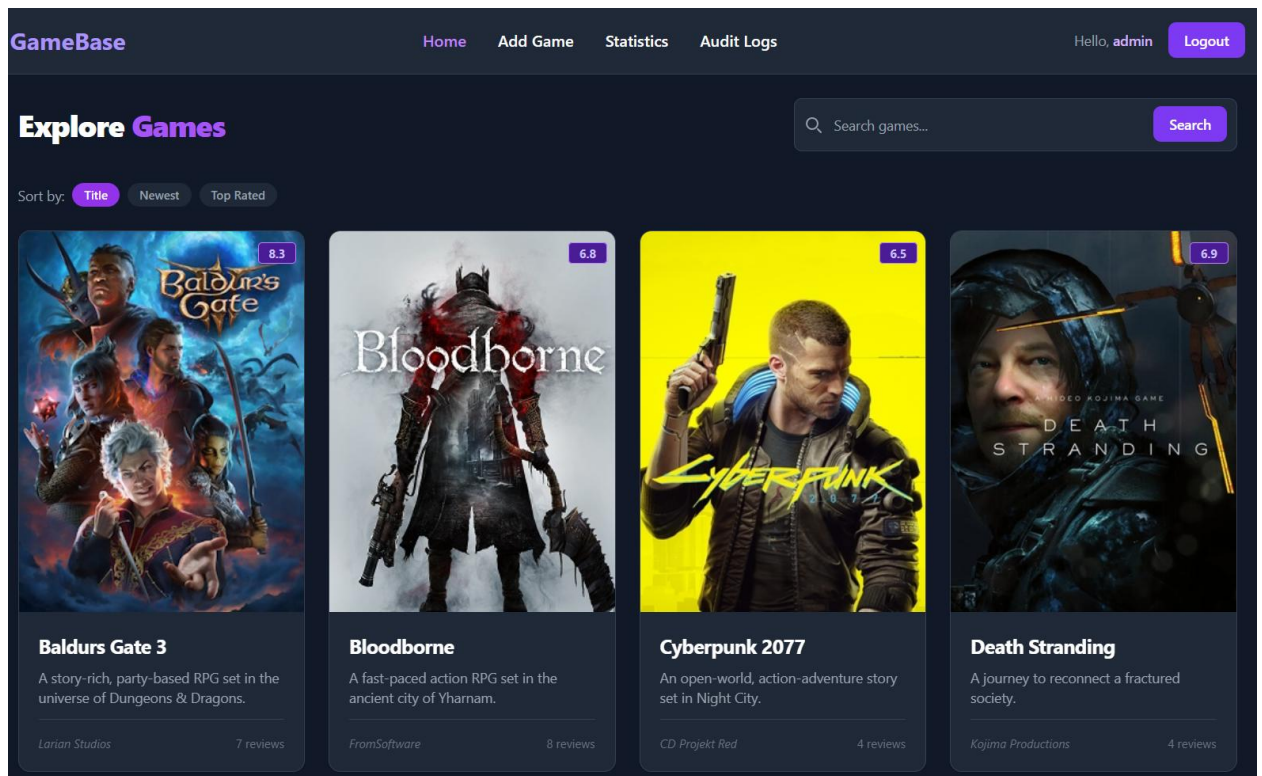


Рисунок 2.10 – Інтерфейс головної сторінки ігрової інформаційної системи.

Головна сторінка містить адаптивну сітку карток відеоігор з відображенням їхніх обкладинок, назв, розробників, дати виходу та середнього рейтингу, який динамічно розраховується з бази даних. У верхній частині розташовано навігаційну панель із кнопками авторизації, пошуковий рядок для фільтрації ігор у реальному часі та випадаючий список для сортування за алфавітом, датою виходу або рейтингом.

На рисунку 2.11 показано інтерфейс детальної сторінки конкретної гри та форму відправки рецензії авторизованим користувачем.

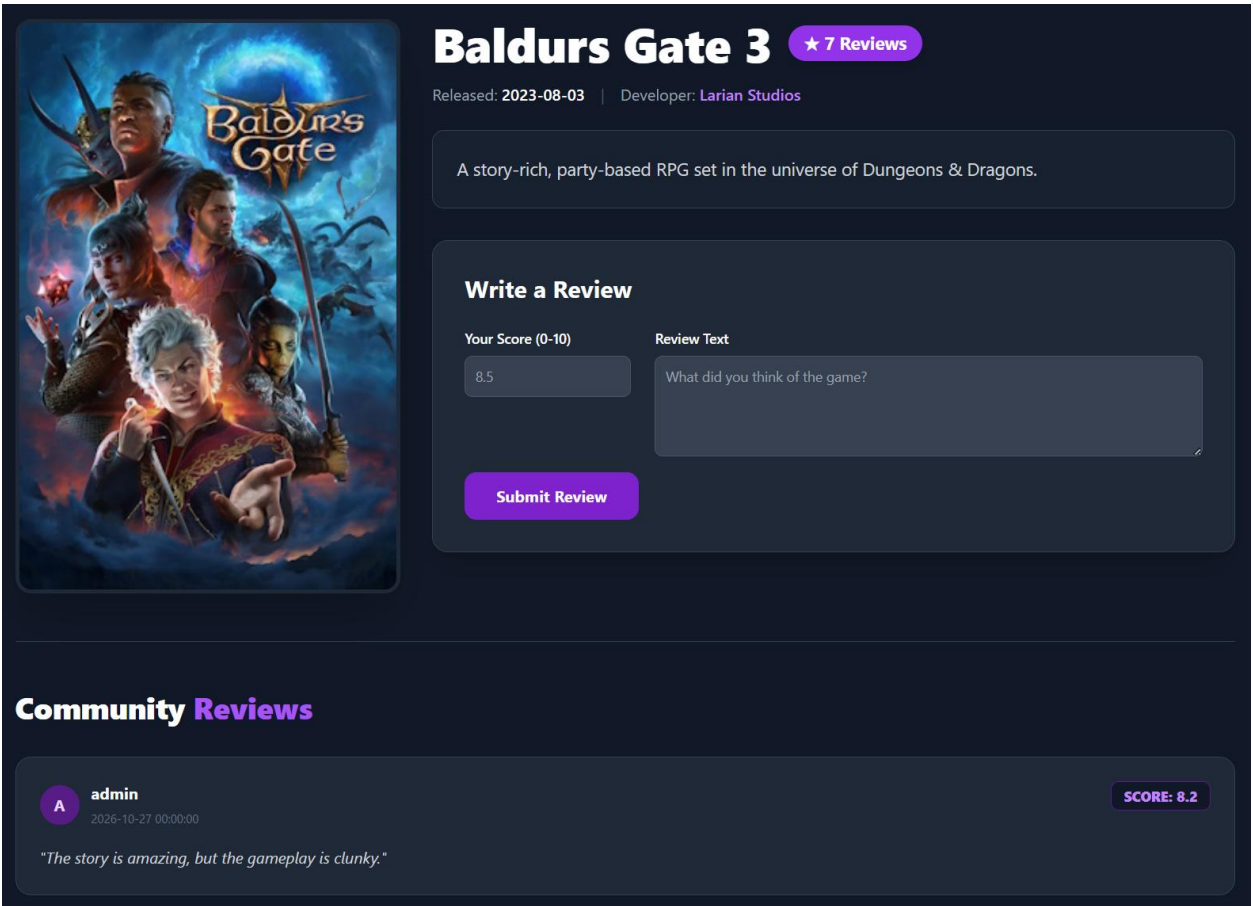


Рисунок 2.11 – Сторінка перегляду деталей гри та додавання відгуку.

На сторінці гри відображається детальний опис проекту, інформація про розробника, а також список рецензій, залишених користувачами системи. Авторизовані користувачі мають доступ до інтерактивної форми оцінювання, де за допомогою слайдера або числового поля виставляють оцінку від 0 до 10 та пишуть текст рецензії, яка після перевірки на унікальність записується в базу даних.

На рисунку 2.12 представлено сторінку динамічної статистики ігрового порталу.

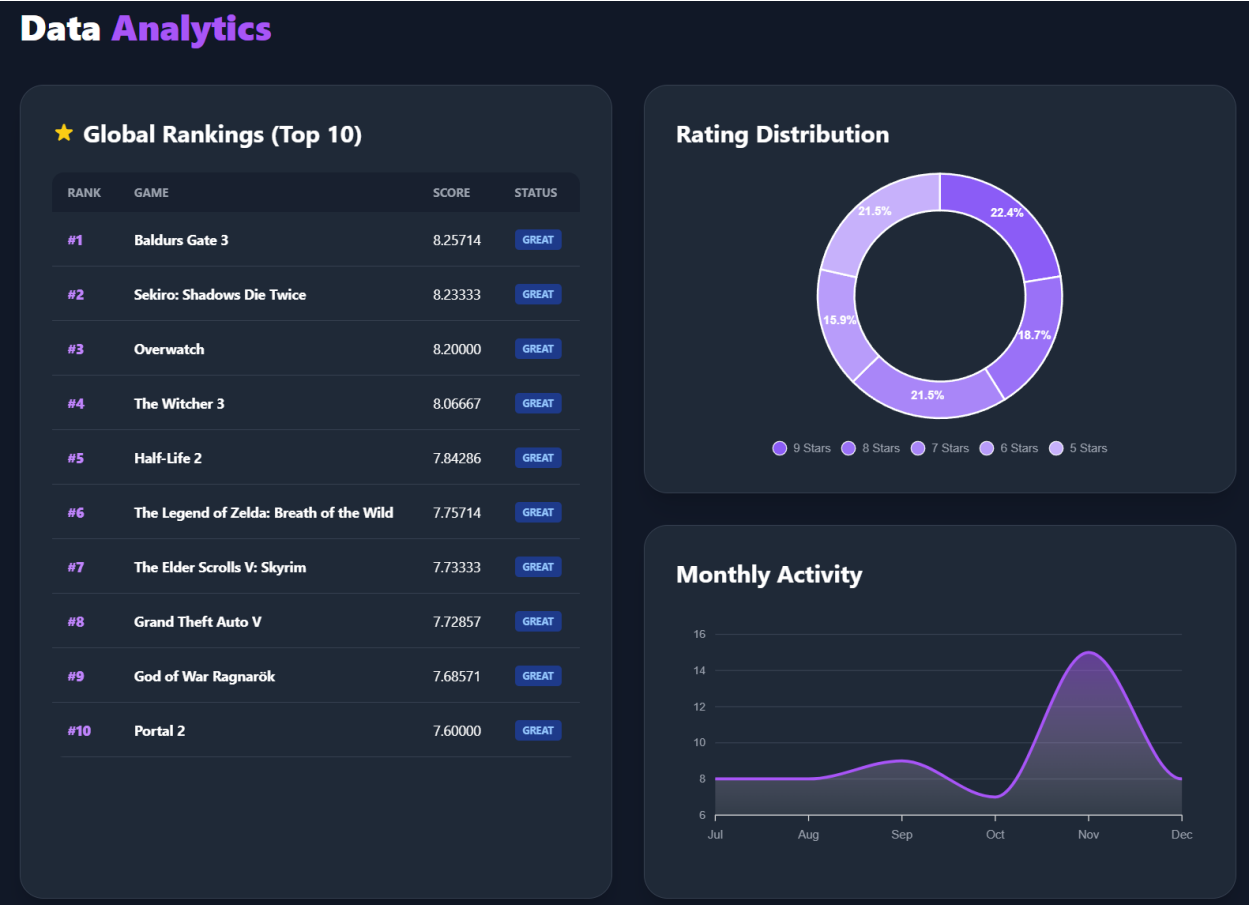


Рисунок 2.12 – Панель аналітичної статистики бази даних.

Сторінка статистики містить інтерактивну діаграму топ-10 ігор за рейтингом, побудовану на основі SQL-представлення `v_game_rankings` та бібліотеки `ApexCharts`. Також виводиться лінійний графік активності користувачів (динаміка додавання відгуків за останні місяці) та стовпчикова діаграма розподілу виставлених оцінок, що дозволяє візуально аналізувати накопичену інформацію.

На рисунку 2.13 зображено адміністративну панель перегляду логів системного аудиту.

System Audit Logs

Trigger-based monitoring active

10 entries per page

Search...

ID	Action	Score (Old → New)	Context	Timestamp
#107	INSERT	8.8	User ID: 4	2026-05-20 21:45:03
#106	INSERT	6.4	User ID: 9	2026-05-20 21:45:03
#105	INSERT	6.3	User ID: 3	2026-05-20 21:45:03
#104	INSERT	6.1	User ID: 10	2026-05-20 21:45:03
#103	INSERT	9.7	User ID: 10	2026-05-20 21:45:03
#102	INSERT	7.2	User ID: 4	2026-05-20 21:45:03
#101	INSERT	9.5	User ID: 1	2026-05-20 21:45:03
#100	INSERT	7.4	User ID: 6	2026-05-20 21:45:03
#99	INSERT	6.4	User ID: 2	2026-05-20 21:45:03
#98	INSERT	9.0	User ID: 7	2026-05-20 21:45:03

Showing 1 to 10 of 20 entries

1 2 >

Рисунок 2.13 – Панель моніторингу логів аудиту адміністратора.

Панель логів є доступною виключно для облікових записів з правами адміністратора. Вона відображає записи з таблиці review_audit_log, які автоматично генеруються тригером бази даних при кожній спробі додати або видалити відгук. Завдяки Simple-DataTables адміністратор може здійснювати миттєвий пошук по логах за ID користувача або типом дії, а також використовувати пагінацію для зручного перегляду великої кількості записів.

ВИСНОВКИ

У ході виконання курсової роботи було успішно спроектовано та програмно реалізовано ігрову інформаційну систему «Game Review System». Для розробки серверної частини веб-застосунку було застосовано мову програмування Python та мікрофреймворк Flask, що дозволило побудувати надійну логіку обробки запитів та маршрутизації сторінок. Зовнішній інтерфейс користувача було оформлено з використанням шаблонізатора Jinja2, фреймворку Tailwind CSS та бібліотеки Flowbite, що забезпечило адаптивність та сучасний візуальний вигляд інтерфейсу. Візуалізацію аналітичних звітів було виконано за допомогою інтерактивної бібліотеки ApexCharts, а пошук по системних журналах реалізовано засобами SimpleDataTables.

Для збереження інформації було обрано реляційну СУБД MySQL. У процесі порівняльного аналізу із NoSQL рішеннями MongoDB та CouchDB було підтверджено перевагу MySQL для задач з чітко структурованими даними. На відміну від нереляційних СУБД, які не забезпечують цілісність даних на рівні бази даних, у розробленій системі було впроваджено суворий контроль цілісності через механізм зовнішніх ключів та каскадних правил оновлення і видалення. Це дозволило повністю усунути ризик появи несуперечливих записів.

Проектування схеми даних було проведено з дотриманням вимог третьої нормальної форми, що мінімізувало надлишковість інформації та оптимізувало обсяг сховища. Завдяки впровадженню збережених процедур було забезпечено атомарність операцій додавання ігор до бази даних. Застосування тригерів дозволило реалізувати автоматичний аудит дій користувачів на рівні ядра бази даних, що підвищило безпеку та надійність системи. Використання віконних SQL-функцій дало змогу автоматизувати

розрахунок рейтингів відеоігор у реальному часі без перевантаження сервера додатків.

Перспективним напрямком розвитку та подальшого вдосконалення проекту є впровадження інтелектуальної системи рекомендацій ігор на основі індивідуальних уподобань користувачів. Також є доцільною реалізація технології кешування даних (наприклад, Redis) для прискорення обробки запитів до популярного контенту та розробка повноцінного RESTful API для можливості масштабування системи та підключення мобільних клієнтів.

ПЕРЕЛІК ПОСИЛАНЬ

- [1] Офіційна документація Tailwind CSS. URL:
<https://tailwindcss.com/docs> (дата звернення: 19.05.2026).
- [2] Офіційна документація бібліотеки Flowbite. URL:
<https://flowbite.com/docs/> (дата звернення: 20.05.2026).
- [3] Офіційна документація СУБД MySQL. URL:
<https://dev.mysql.com/doc/> (дата звернення: 15.05.2026).
- [4] Офіційна документація NoSQL СУБД MongoDB. URL:
<https://www.mongodb.com/docs/> (дата звернення: 16.05.2026).
- [5] Офіційна документація Apache CouchDB. URL:
<https://docs.couchdb.org/> (дата звернення: 17.05.2026).
- [6] Офіційна документація веб-фреймворку Flask. URL:
<https://flask.palletsprojects.com/> (дата звернення: 18.05.2026).
- [7] Курс Cisco Networking Academy: Python Essentials 1. URL:
<https://www.netacad.com/> (дата звернення: 20.05.2026).

ДОДАТОК А

Програмна реалізація бази даних

```
-- Пересоздание базы данных
DROP DATABASE IF EXISTS game_db;
CREATE DATABASE game_db;
USE game_db;

-- Таблица разработчиков игр
CREATE TABLE IF NOT EXISTS developers (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(150) NOT NULL UNIQUE,
    country VARCHAR(100) NOT NULL
);

-- Таблица издателей игр
CREATE TABLE IF NOT EXISTS publishers (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(150) NOT NULL UNIQUE,
    country VARCHAR(100) NOT NULL
);

-- Справочник жанров
CREATE TABLE IF NOT EXISTS genres (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE
);

-- Справочник игровых платформ
CREATE TABLE IF NOT EXISTS platforms (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
    abbreviation VARCHAR(20) NOT NULL UNIQUE
);

-- Таблица пользователей системы
CREATE TABLE IF NOT EXISTS users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(100) NOT NULL UNIQUE,
    email VARCHAR(255) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    is_admin TINYINT DEFAULT 0,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Основная таблица игр
CREATE TABLE IF NOT EXISTS games (
    id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(255) NOT NULL UNIQUE,
    release_date DATE NOT NULL,
    description TEXT NOT NULL,
    developer_id INT NOT NULL,
    publisher_id INT NOT NULL,
    cover_url VARCHAR(255),
    FOREIGN KEY (developer_id) REFERENCES developers(id) ON DELETE CASCADE,
    FOREIGN KEY (publisher_id) REFERENCES publishers(id) ON DELETE CASCADE
);

-- Связующая таблица: Игры <-> Жанры
CREATE TABLE IF NOT EXISTS game_genres (
    game_id INT NOT NULL,
    genre_id INT NOT NULL,
    PRIMARY KEY (game_id, genre_id),
    FOREIGN KEY (game_id) REFERENCES games(id) ON DELETE CASCADE,
    FOREIGN KEY (genre_id) REFERENCES genres(id) ON DELETE CASCADE
);

-- Связующая таблица: Игры <-> Платформы
CREATE TABLE IF NOT EXISTS game_platforms (
    game_id INT NOT NULL,
    platform_id INT NOT NULL,
    PRIMARY KEY (game_id, platform_id),
    FOREIGN KEY (game_id) REFERENCES games(id) ON DELETE CASCADE,
    FOREIGN KEY (platform_id) REFERENCES platforms(id) ON DELETE CASCADE
);
```

```

);

-- Таблица отзывов пользователей
CREATE TABLE IF NOT EXISTS reviews (
    id INT AUTO_INCREMENT PRIMARY KEY,
    game_id INT NOT NULL,
    user_id INT NOT NULL,
    score DECIMAL(4,1) NOT NULL CHECK (score >= 0.0 AND score <= 10.0),
    review_text TEXT NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY uq_reviews_game_user (game_id, user_id),
    FOREIGN KEY (game_id) REFERENCES games(id) ON DELETE CASCADE,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

-- Таблица логов аудита
CREATE TABLE IF NOT EXISTS review_audit_log (
    id INT AUTO_INCREMENT PRIMARY KEY,
    review_id INT,
    action_type ENUM('INSERT', 'UPDATE', 'DELETE'),
    action_timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
    old_score DECIMAL(4,1),
    new_score DECIMAL(4,1),
    user_info VARCHAR(255)
);

-- Функция статуса оценки
DELIMITER //
CREATE FUNCTION fn_get_rating_status(score DECIMAL(4,1))
RETURNS VARCHAR(20)
DETERMINISTIC
BEGIN
    DECLARE status VARCHAR(20);
    IF score >= 9.0 THEN SET status = 'Masterpiece';
    ELSEIF score >= 7.5 THEN SET status = 'Great';
    ELSEIF score >= 5.0 THEN SET status = 'Mediocre';
    ELSE SET status = 'Poor';
    END IF;
    RETURN status;
END //
DELIMITER ;

-- Представление рейтингов игр (Включает оконную функцию RANK)
CREATE OR REPLACE VIEW v_game_rankings AS
SELECT
    g.id,
    g.title,
    d.name as developer,
    p.name as publisher,
    COALESCE(avg_rev.score, 0) as total_score,
    fn_get_rating_status(COALESCE(avg_rev.score, 0)) as rating_status,
    RANK() OVER (ORDER BY COALESCE(avg_rev.score, 0) DESC) as global_rank
FROM games g
JOIN developers d ON g.developer_id = d.id
JOIN publishers p ON g.publisher_id = p.id
LEFT JOIN (SELECT game_id, AVG(score) as score FROM reviews GROUP BY game_id) avg_rev ON g.id =
avg_rev.game_id;

-- Хранимая Процедура: Добавление игры
DELIMITER //
CREATE PROCEDURE sp_add_game_full(
    IN p_title VARCHAR(255),
    IN p_release_date DATE,
    IN p_description TEXT,
    IN p_developer_id INT,
    IN p_publisher_id INT,
    IN p_cover_url VARCHAR(255)
)
BEGIN
    INSERT INTO games (title, release_date, description, developer_id, publisher_id, cover_url)
    VALUES (p_title, p_release_date, p_description, p_developer_id, p_publisher_id, p_cover_url);

    SELECT LAST_INSERT_ID() AS game_id;
END //
DELIMITER ;

```

```
-- Триггер: Автоматическое логирование отзывов
DELIMITER //
CREATE TRIGGER tr_after_review_insert
AFTER INSERT ON reviews
FOR EACH ROW
BEGIN
    INSERT INTO review_audit_log (review_id, action_type, new_score, user_info)
    VALUES (NEW.id, 'INSERT', NEW.score, CONCAT('User ID: ', NEW.user_id));
END //
DELIMITER ;
```